

Waypoint Frontend Architecture

Complete Source Code Walkthrough, React Flow DAG Engine & Engineering Reference

Written for 1st-Year B.Tech IT Students to Independently Understand, Maintain, Debug & Extend the System

Review 2 Milestone • Next.js 14 App Router • TypeScript • React Flow • Tailwind CSS

Pedagogical Objective: This manual provides an exhaustive, line-by-line conceptual and structural explanation of the entire Waypoint frontend. It details how routing, custom graph nodes, state management, and the API layer work together, explicitly identifying where real mock simulation is active versus where FastAPI endpoints will be attached in the next phase.

Document Outline & Syllabus

Part 1: Complete Directory Tree & Map	Structure & Flow	Part 2: Next.js 14 App Router Architecture	Server vs Client
Part 3: End-to-End User Journey Trace	11 Full Steps	Part 4: Major Component Deep Dives	Props, Hooks & State
Part 5: React Flow DAG Graph Engine	Topological Layering	Part 6: API Layer & Endpoint Audit	Contract Matrix
Part 7: TypeScript Domain Types	Contracts & Generics	Part 8: State Management Architecture	Context & Storage
Part 9: Data Flow & Mock Data Inventory	Storage Boundaries	Part 10: Styling & Design System	Tailwind & Tokens
Part 11: Real Implementation Status	4 Honest Categories	Part 12: Senior Code Quality Review	Severity Audit
Part 13: Recommended Learning Roadmap	Ordered Curriculum	Part 14: Progressive Code Walkthrough	From Utils to DAG
Part 15: Master System Architecture Map	End-to-End Diagram		

01 Complete Frontend Directory Structure

The Waypoint frontend is structured under `frontend/` following modern Next.js 14 conventions. Every file has a single, well-defined responsibility, ensuring maintainability as the project scales.

```

frontend/
├── app/
│   ├── layout.tsx      # Next.js App Router (Routing, Layouts, & Viewports)
│   ├── page.tsx        # Root Layout: HTML shell, Inter font, AuthProvider, globals.css
│   ├── loading.tsx     # Public Landing Page: Hero, Value Proposition, How It Works
│   ├── error.tsx       # Automated Suspense Fallback during route transitions
│   ├── not-found.tsx   # Global Client-side Error Boundary with retry handler
│   ├── login/page.tsx  # 404 Catch-all View
│   ├── signup/page.tsx # Student Authentication & Login Form
│   ├── onboarding/page.tsx # Student Registration Form
│   ├── dashboard/page.tsx # 4-Step Academic & Goal Selection Wizard
│   ├── routes/page.tsx # Main Intelligence Hub (ProgressRing, Journey, NextStep)
│   └── route/[id]/
│       ├── page.tsx    # 3-Way Route Comparison Screen (Balanced, Fast, Foundation)
│       └── RouteMapClient.tsx # Dynamic Route Visualization Segment
│
├── components/
│   ├── auth/AuthGuard.tsx # Server Component route wrapper
│   ├── crowd-trails/CrowdTrails.tsx # Client dynamic boundary (ssr: false for React Flow)
│   ├── dashboard/
│   │   ├── JourneyCard.tsx # Categorized Skill Deficit Audit (Critical, High, Recommended)
│   │   ├── JourneyProgress.tsx # Action Ecosystem (Hackathons, Internships, Research Labs)
│   │   ├── NextStepCard.tsx # Modular, Reusable React Components
│   │   ├── RouteCardPreview.tsx # Route Protection Gatekeeper (checks session token)
│   │   └── SkillGapSummary.tsx # Senior Alumni Trajectory & Course Path Visualizer (Scaffolded)
│   ├── landing/
│   │   ├── HeroRouteViz.tsx # Dashboard Overview Widgets
│   │   ├── HeroSection.tsx # Target Goal Card with SVG Circular Progress Ring
│   │   └── HowItWorks.tsx # Horizontal Timeline Progression Strip
│   ├── layout/
│   │   ├── AppShell.tsx # High-Priority Recommended Immediate Action Banner
│   │   ├── Sidebar.tsx # Mini Route Metric Summary Card
│   │   └── Topbar.tsx # Priority-Segmented Deficit Breakdown Box
│   ├── onboarding/
│   │   ├── ProfileStep.tsx # Landing Page Components
│   │   ├── AcademicStep.tsx # Animated SVG Mini-Graph Preview
│   │   ├── SkillsStep.tsx # Landing Headline & Primary Call-to-Action
│   │   ├── DestinationStep.tsx # 3-Step Systematic Explanation Grid
│   │   └── ProgressStepper.tsx # Structural Application Shell
│   ├── opportunities/
│   │   ├── OpportunitiesList.tsx # Unified Master Layout (Sidebar + Topbar + Content)
│   │   └── route-map/
│   │       ├── RouteMap.tsx # Fixed 224px Left Rail with Active Path Highlights
│   │       ├── SkillNode.tsx # Top Header with Breadcrumbs & Profile Dropdown
│   │       ├── SkillDetailPanel.tsx # Onboarding Wizard Steps
│   │       └── MapLegend.tsx # Step 1: Name, Degree Program, & Semester
│   ├── routes/RouteCard.tsx # Step 2: CGPA & Completed Credits with Visual Standing
│   ├── skill-gaps/
│   │   ├── SkillCard.tsx # Step 3: Current Acquired Skills Tag Cloud
│   │   └── SkillGapSection.tsx # Step 4: Career Role Selection Cards
│   └── ui/
│       ├── Button.tsx # 4-Stage Visual Progress Bar
│       ├── Badge.tsx # Opportunity Radar UI
│       ├── Card.tsx # Filterable, Searchable Opportunity Card Feed
│       ├── ProgressBar.tsx # Interactive React Flow DAG Engine
│       ├── ProgressRing.tsx # Canvas Container, Controls, MiniMap, & State
│       ├── SkeletonLoader.tsx # Custom React Flow Node Component (Handles & Status)
│       ├── EmptyState.tsx # Slide-Over Drawer with Curated Study Resources
│       └── ErrorState.tsx # Floating Visual Status & Category Color Legend
│
├── contexts/AuthContext.tsx # Detailed Route Metric Card with Step Timeline
├── hooks/
│   ├── useRoute.ts # Skill Deficit Breakdown UI
│   └── useStudentProfile.ts # Interactive Skill Card with Expandable Why-It-Matters
├── lib/
│   ├── api.ts # Section Container grouped by Priority Level
│   ├── types.ts # Design System Primitives
│   ├── utils.ts # Polymorphic Button with loading spinner and icon slots
│   └── mock-data/
│       ├── catalog.ts # Semantic Status Badges (success, warning, danger, info)
│       ├── destinations.ts # Accessible Container with hover elevation
│       ├── opportunities.ts # Linear Animated Progress Bar
│       ├── routes.ts # SVG Radial Progress Indicator
│       ├── skills.ts # Shimmer Loading Placeholders
│       └── student.ts # Zero Search Results / Empty List Display
├── styles/globals.css # API Failure State with "Try Again" trigger
├── tailwind.config.ts # React Context for Global Authentication & User Session
├── tsconfig.json # Custom React Hooks
├── package.json # Core Domain Logic & Data Layer
└── 
```

System Dependency Flow Diagram

```

[ User Browser (Interactions, Clicks, Keystrokes) ]
|
↓
[ Next.js App Router (app/layout.tsx) ]
|
├── [ AuthProvider (contexts/AuthContext.tsx) ] → [ localStorage (Session/Accounts) ]
|
↓
[ Page Views (app/**/page.tsx) ]
|
├── Protected by [ AuthGuard (components/auth/AuthGuard.tsx) ]
|
├── Wrapped in [ AppShell (components/layout/AppShell.tsx) ]
|   ├── [ Sidebar.tsx ] (Nav Links & Framer Motion indicator)
|   └── [ Topbar.tsx ] (Breadcrumbs & Profile avatar)
|
├── Feature Components:
|   ├── [ RouteMap.tsx ] → React Flow Canvas
|   │   ├── [ SkillNode.tsx ] (Custom Handles & Status styling)
|   │   └── [ SkillDetailPanel.tsx ] (Slide-over drawer)
|   ├── [ JourneyCard.tsx ] → SVG ProgressRing
|   ├── [ SkillGapSection.tsx ] → [ SkillCard.tsx ]
|   └── [ OpportunitiesList.tsx ] → Filterable Opportunity Cards
|
↓
[ Central API Layer (frontend/lib/api.ts) ]
|
├── (Review 2 Demo Mode): [ lib/mock-data/*.ts ] + Browser localStorage
|
└── (Future Review 3 Milestone)
[ FastAPI Backend (`http://localhost:8000/api/v1/...`) ]

```

02 Next.js Architecture Explained

Waypoint is built with **Next.js 14** using the **App Router** paradigm. Rather than configuring complex routing files like in standard React (React Router), Next.js maps the folder hierarchy inside `frontend/app/` directly to URL routes.

Core App Router Conventions in Waypoint

- `layout.tsx` : The root wrapper. Runs on initial load to set up `<html>`, `<body>`, mounts global CSS, and wraps everything in `<AuthProvider>`. It persists state across page changes without re-mounting.
- `page.tsx` : Represents the publicly accessible leaf component for that URL segment.
- `loading.tsx` : Built on React `<Suspense>`. Next.js automatically shows this centered spinner while data is resolving during route transitions.
- `error.tsx` : Global client-side error boundary (must declare `'use client'`). Catches unhandled runtime exceptions in child components and renders a friendly retry screen without crashing the whole tab.
- `not-found.tsx` : Default 404 handler rendered when an invalid URL is requested.

Server vs. Client Components (`'use client'`)

By default, every component in Next.js 14 is a **Server Component**. Server Components execute solely on the server, generating zero JavaScript in the client's browser bundle.

However, whenever a component needs:

- Browser APIs (e.g., `localStorage`, `window`, `document`)
- React State or Lifecycle hooks (`useState`, `useEffect`, `useCallback`)
- User interactivity (e.g., `onClick`, `onChange`, `onSubmit`)
- Third-party canvas manipulation (e.g., `React Flow`)

It must declare `'use client'` at the very top of the file. In Waypoint, interactive viewports (like Dashboard, Onboarding, and Opportunities) are Client Components.

Special Architectural Pattern: `RouteMapClient.tsx`

React Flow relies heavily on browser DOM measurement (calculating bounding boxes for SVG bezier curves). During Next.js server-side pre-rendering, Node.js has no DOM, which causes hydration mismatches or server crashes.

Waypoint elegantly solves this in `frontend/app/route/[id]/RouteMapClient.tsx` :

```
import dynamic from 'next/dynamic'
import { SkeletonCard } from '@components/ui/SkeletonLoader'

export const RouteMap = dynamic(
  () => import('@components/route-map/RouteMap').then((m) => m.RouteMap),
  {
    ssr: false, // Disables server pre-rendering for the canvas!
    loading: () => <div className="h-full flex items-center justify-center"><SkeletonCard /></div>,
  }
)
```

Route Navigation Matrix

URL Route	Source File	Component Type	What Happens Upon Navigation
<code>/</code>	<code>app/page.tsx</code>	Client Component	Renders marketing landing page, animated hero graph, and "Get Started" call-to-actions.
<code>/login</code>	<code>app/login/page.tsx</code>	Client Component	Renders sign-in form. Calls <code>authLogin()</code> , stores session in <code>localStorage</code> , pushes to <code>/dashboard</code> .
<code>/signup</code>	<code>app/signup/page.tsx</code>	Client Component	Renders registration form. Creates account credentials, sets session, and routes to <code>/onboarding</code> .
<code>/onboarding</code>	<code>app/onboarding/page.tsx</code>	Client Component	4-step wizard. Saves completed <code>StudentProfile</code> to <code>localStorage</code> and routes to <code>/dashboard</code> .
<code>/dashboard</code>	<code>app/dashboard/page.tsx</code>	Client Component	Protected by <code>AuthGuard</code> . Loads student profile, calculates progress %, displays next actionable step and opportunity radar.
<code>/routes</code>	<code>app/routes/page.tsx</code>	Client Component	Loads all 3 generated routes and renders the side-by-side comparison table (Balanced, Fast Track, Foundation First).
<code>/route/[id]</code>	<code>app/route/[id]/page.tsx</code>	Server/Client Hybrid	Extracts route ID slug (e.g. <code>balanced</code>), mounts React Flow client canvas without SSR, and renders interactive skill graph.
<code>/skill-gaps</code>	<code>app/skill-gaps/page.tsx</code>	Client Component	Computes missing competencies against target destination requirements; groups them into Critical, High, and Recommended.
<code>/opportunities</code>	<code>app/opportunities/page.tsx</code>	Client Component	Renders actionable hackathons, internships, and research grants matching the student's selected branch and destination.

03 Trace the Application From Start to Finish

To understand how data flows across the system, let us trace the complete journey of a new student using Waypoint from initial visit to opportunity application.

Step 1: User Opens Website & Views Landing Page

- **Files:** `frontend/app/layout.tsx` → `frontend/app/page.tsx` → `HeroSection.tsx`
- **Component:** `LandingPage()`, `HeroSection()`, `HeroRouteViz()`
- **State:** Framer Motion entrance animation states (`opacity: 0, y: 24` → `opacity: 1, y: 0`).
- **Data:** Static SVG visual graph showing nodes (You, Python, Linear Algebra, Data Structures, Machine Learning).
- **API Call:** None. Static marketing presentation. Clicking "Start Your Journey" routes to `/onboarding`.

Step 2: User Signs Up / Authenticates

- **Files:** `frontend/app/signup/page.tsx` → `frontend/contexts/AuthContext.tsx`
- **Function:** `handleSignup(e: React.FormEvent)` invokes `signup()` from `useAuth()`.
- **API Invocation:** `authSignup(name, email, password)` in `frontend/lib/api.ts`.
- **SIMULATED IN BROWSER STORAGE:** Validates email uniqueness, creates an account object, encodes credentials via `btoa`, writes to `localStorage.setItem('waypoint_accounts', ...)` and `localStorage.setItem('waypoint_session', ...)`, then pushes router to `/onboarding`.

Step 3: User Completes 4-Step Onboarding

- **File:** `frontend/app/onboarding/page.tsx`
- **Component:** `OnboardingPage()` with sub-components:
 1. `ProfileStep.tsx`: Captures `name`, `program` (e.g., B.Tech IT), and `semester`.
 2. `AcademicStep.tsx`: Captures `cgpa` (e.g., 8.2) and `creditsCompleted` (60). Computes dynamic standing badge.
 3. `SkillsStep.tsx`: Multi-select pill cloud toggling acquired skills into `currentSkills` array.
 4. `DestinationStep.tsx`: Calls `getDestinations()` and renders selectable `DestinationCard` components.
- **Final Submission:** Calls `createStudentProfile(finalProfile)` in `lib/api.ts`.
- **SIMULATED IN BROWSER STORAGE:** Commits profile JSON to `localStorage.setItem('waypoint_profile', ...)` and redirects to `/dashboard`.

Step 4: User Reaches Dashboard Overview

- **File:** `frontend/app/dashboard/page.tsx` (enclosed by `AuthGuard` and `AppShell`).
- **API Invocation:** `Promise.all([getStudentProfile(), getRoutes()])`.
- **Execution:** Resolves profile from `localStorage` (or `mockStudent`). Computes progress: `progress = Math.round((completedSkills / totalRequiredSkills) * 100)`
- **Components Rendered:** `JourneyCard` (SVG `ProgressRing`), `JourneyProgress` (sequential milestone nodes), `NextStepCard`, and `RouteCardPreview`.

Step 5: User Inspects Skill Gaps

- **File:** `frontend/app/skill-gaps/page.tsx`
- **API Invocation:** `getSkillGaps(profile.currentSkills, profile.selectedDestinationId)`.
- **Execution:** Calculates set difference between destination requirements and student skills. Categorizes missing skills into **Critical** (immediate prerequisites), **High Priority**, and **Recommended**.
- **Component:** `SkillCard.tsx` renders expandable details explaining why the skill matters and downstream dependencies.

Step 6: User Explores & Interacts with Route Map

- **Files:** `app/route/[id]/page.tsx` → `components/route-map/RouteMap.tsx`
- **API Invocation:** `getRouteGraph(routeId, profile.selectedDestinationId, destinationTitle, profile.currentSkills)`.
- **Algorithm:** `generateDestinationGraph()` computes topological depth for each skill, positions nodes vertically by layer ($y = 20 + \text{ext}(\text{depth}) \text{ times } 130$) and horizontally centered across $X=340$, connecting prerequisites via smoothstep orthogonal bezier edges.
- **Interactions:**
 - Pan & Zoom canvas via mouse wheel/drag.
 - Clicking "Focus Next Skill" calls `reactFlowInstance.setCenter(100, 220, { zoom: 1.2, duration: 600 })`.
 - Clicking any skill node opens `SkillDetailPanel.tsx` drawer displaying curated study resources.

Step 7: User Discovers Opportunities

- **File:** `frontend/app/opportunities/page.tsx`
- **API Invocation:** `getOpportunities(destinationId, activeType, filterMode)`.
- **Execution:** Returns real-world contests (Smart India Hackathon, Google Summer of Code, DRDO Fellowships) matched to the student's career target. Users can filter by category pills, search by keywords, bookmark, or click "Apply Now".

04 Major Component Deep Dives

1. components/route-map/RouteMap.tsx (Graph Engine)

- **Role:** Hosts the React Flow canvas, top floating control bar, mini-map, and slide-out detail drawer.
- **Props:** `{ routeId: string }`
- **State Hooks:**
 - `[nodes, setNodes, onNodesChange] = useNodesState([])` : React Flow node array and change listener.
 - `[edges, setEdges, onEdgesChange] = useEdgesState([])` : React Flow edge array and change listener.
 - `selectedSkill: Skill | null` : Currently clicked skill; opens slide-out drawer when non-null.
 - `loading: boolean, error: string | null` : Async status flags.
- **Event Handlers:**
 - `onNodeClick` : Reads `node.data.skillId`, finds the skill, and populates `selectedSkill`.
 - `onPaneClick` : Deselects the active skill and closes the slide-out drawer.
 - `handleCenterOnNext` : Smoothly animates camera to target coordinates using `useReactFlow()`.

2. components/route-map/SkillNode.tsx (Custom Node)

- **Role:** Custom visual component rendered by React Flow at exact (x, y) coordinates.
- **Props:** Standard `NodeProps<RouteNodeData>`.
- **Branching Logic:**
 - `data.isStart === true` : Renders a dark navy Origin badge with an emerald compass icon and a bottom source handle.
 - `data.isGoal === true` : Renders a target destination card with an amber trophy icon and a top target handle.
 - **Standard Skill Node:** Renders a 165px card with a category badge, status icon (Green Check for completed, Blue Play for next, Grey Lock for locked), and two `<Handle />` connection anchors at top and bottom.
- **Optimization:** Wrapped in `React.memo` to prevent unnecessary re-rendering while panning the canvas.

3. components/dashboard/JourneyCard.tsx

- **Role:** Primary summary card at the top of the student dashboard.
- **Props:** `{ destination: string; progress: number; completedSkills: number; totalSkills: number }`
- **Rendering:** Displays the target destination name with a location pin, acquired vs remaining skill counts, and renders an SVG radial `<ProgressRing value={progress} size={88} />`.

4. components/layout/Sidebar.tsx

- **Role:** Fixed 224px (w-56) left navigation rail.
- **Routing Logic:** Uses Next.js `usePathname()` hook to compare the current URL against navigation item links (`/dashboard`, `/route/balanced`, `/skill-gaps`, `/routes`).
- **Animation:** Uses Framer Motion's `layoutId="sidebar-indicator"` to glide an active blue indicator pill between links as the user navigates.

5. components/opportunities/OpportunitiesList.tsx

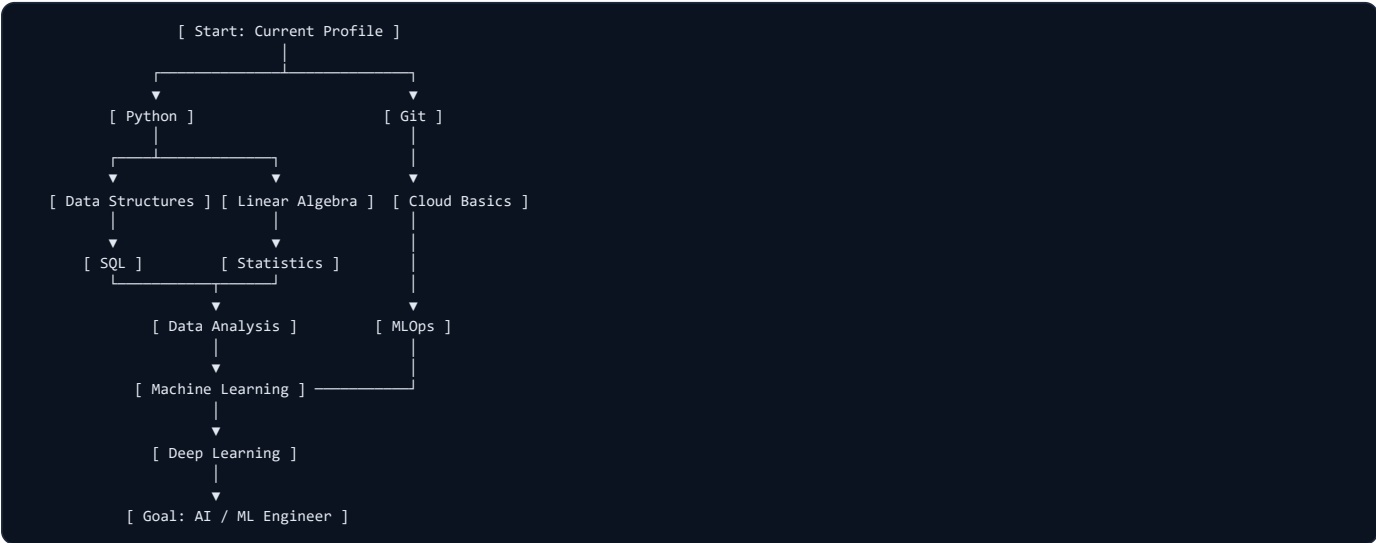
- **Role:** Searchable and filterable directory of contests, internships, and grants.
- **State:**
 - `activeTab` : `'all' | 'trending' | 'closing-soon' | 'new-today'`
 - `activeType` : `'all' | 'Hackathon' | 'Internship' | 'Research' | 'Incubator'`
 - `searchQuery` : String filter for keyword searching on titles, organizations, and tags.
 - `bookmarkedIds` : Set of bookmarked opportunity IDs persisted in local component state.
- **Derived Filtering:**

```
const filtered = opportunities.filter((op) => {
  const matchesSearch = op.title.toLowerCase().includes(query) ||
    op.organization.toLowerCase().includes(query)
  const matchesType = activeType === 'all' || op.type === activeType
  return matchesSearch && matchesType
})
```

05

React Flow DAG Graph Implementation

React Flow transforms academic course prerequisites into an interactive, visual **Directed Acyclic Graph (DAG)**. Understanding this subsystem is critical for maintaining Waypoint's core feature.



1. Nodes, Edges, and Custom Types

- **Nodes (`Node<RouteNodeData>`)**: Represent skills. Contains an `id`, `type: 'skillNode'`, canvas coordinates `{ x, y }`, and a custom data payload (title, category, status, priority).
- **Edges (`Edge`)**: Represent prerequisite connections. Specifies `source`, `target`, and `type: 'smoothstep'`.
- **Handles (`<Handle />`)**: The anchor coordinates on the custom node where edge paths attach:
 - `<Handle type="target" position={Position.Top} />` : Where prerequisite edges enter.
 - `<Handle type="source" position={Position.Bottom} />` : Where downstream dependent edges leave.

2. Topological Layering Algorithm (`generateDestinationGraph`)

Located in `frontend/lib/mock-data/routes.ts`, this algorithm organizes an arbitrary collection of prerequisites into a clean, collision-free vertical layout:

1. **Depth Calculation**: Recursively determines the longest chain of prerequisites for every skill: $\text{depth}(S) = 1 + \max_{P \in \text{Prerequisites}(S)} \text{depth}(P)$
Circular dependencies are guarded using a `visited` Set.
2. **Vertical Slicing**: Skills with the same depth are grouped into horizontal layers. Vertical coordinate is calculated as: $Y = 20 + \text{depth} \times 130 \text{px}$
3. **Horizontal Centering**: Skills in each layer are centered symmetrically around canvas midpoint $X = 340$: $X = 340 - \frac{(N - 1) \times \text{spacing}}{2} + (i \times \text{spacing})$
4. **Goal Anchoring**: Terminal skills with no downstream dependents connect directly into the final `goal` node anchored at the bottom: $Y_{\text{goal}} = 20 + (\text{maxDepth} + 1) \times 130 \text{px}$

06 API Architecture & Endpoint Audit

All data operations go through `frontend/lib/api.ts`. The function signatures return standard async Promises. This creates a clean abstraction boundary: when the FastAPI backend is connected, **zero UI component files will need modification**.

Frontend Function	HTTP	Target Endpoint	Payload / Params	Return Type	Calling View	Backend Exists?	Integration Status
<code>getStudentProfile()</code>	GET	<code>/api/student/profile</code>	None	<code>StudentProfile</code>	Dashboard, RouteMap, SkillGaps	NO	Simulated (localStorage)
<code>createStudentProfile()</code>	POST	<code>/api/student/profile</code>	<code>StudentProfile</code>	<code>StudentProfile</code>	OnboardingPage	NO	Simulated (localStorage)
<code>getAllSkills()</code>	GET	<code>/api/skills</code>	<code>destinationId?</code>	<code>Skill[]</code>	SkillGaps, Onboarding	NO	Simulated (mockSkills)
<code>getSkillById(id)</code>	GET	<code>/api/skills/{id}</code>	<code>id: string</code>	<code>Skill null</code>	RouteMap (on node click)	NO	Simulated (mockSkills)
<code>getSkillGaps()</code>	GET	<code>/api/skills/gaps</code>	<code>currentSkillIds, destId</code>	<code>Skill[]</code>	Dashboard, SkillGaps	NO	Simulated (mockSkills)
<code>getDestinations()</code>	GET	<code>/api/destinations</code>	None	<code>Destination[]</code>	DestinationStep	NO	Simulated (mockDestinations)
<code>getDestination(id)</code>	GET	<code>/api/destinations/{id}</code>	<code>id: string</code>	<code>Destination</code>	Dashboard, RouteMap	NO	Simulated (mockDestinations)
<code>getRoutes()</code>	GET	<code>/api/routes</code>	None	<code>Route[]</code>	Dashboard, RoutesPage	NO	Simulated (mockRoutes)
<code>getRoute(id)</code>	GET	<code>/api/routes/{id}</code>	<code>routeId: string</code>	<code>Route null</code>	RoutesPage, RouteMap	NO	Simulated (mockRoutes)
<code>getRouteGraph()</code>	GET	<code>/api/routes/{id}/graph</code>	<code>routeId, destId, skills</code>	<code>{ nodes, edges }</code>	RouteMap	NO	Simulated (generateDAG)
<code>getOpportunities()</code>	GET	<code>/api/opportunities</code>	<code>destId, type, filter</code>	<code>Opportunity[]</code>	OpportunitiesPage	NO	Simulated (mockOpportunities)
<code>authLogin()</code>	POST	<code>/api/auth/login</code>	<code>email, password</code>	<code>AuthUser</code>	LoginPage, AuthContext	NO	Simulated (localStorage)
<code>authSignup()</code>	POST	<code>/api/auth/signup</code>	<code>name, email, password</code>	<code>AuthUser</code>	SignupPage, AuthContext	NO	Simulated (localStorage)
<code>authLogout()</code>	POST	<code>/api/auth/logout</code>	None	<code>void</code>	Sidebar, Topbar	NO	Simulated (localStorage)
<code>getCurrentUser()</code>	GET	Sync Storage Read	None	<code>AuthUser null</code>	AuthContext	N/A	Working (localStorage)

How to Connect FastAPI Later: To wire the frontend to a real backend, replace the functions in `frontend/lib/api.ts` with `fetch()` calls. For example:

```
export async function getStudentProfile(): Promise<StudentProfile> {
  const res = await fetch('http://localhost:8000/api/v1/student/profile', {
    headers: { Authorization: `Bearer ${getToken()}` },
  })
  if (!res.ok) throw new Error('Failed to fetch profile')
  return res.json()
}
```

07 TypeScript Types Deep Dive

All core domain entities are declared in `frontend/lib/types.ts`. Using strict TypeScript contracts prevents runtime bugs by enforcing shape validity at compile time.

1. StudentProfile Entity

```
export type StudentStage =
  | 'just-exploring'
  | 'planning-to-enroll'
  | 'currently-enrolled'
  | 'working-professional'

export interface StudentProfile {
  name: string
  program: string
  branch: string
  studentStage: StudentStage
  semester?: number // Optional: omitted for non-enrolled students
  cgpa?: number // Optional: omitted for non-enrolled students
  creditsCompleted?: number // Optional
  currentSkills: string[] // Array of acquired skill IDs
  selectedDestinationId: string // Matches Destination.id
}
```

- **Union Types (|)**: `StudentStage` restricts values to 4 exact strings. Any typo (e.g. `'enrolled'`) causes a compile-time error.
- **Optional Fields (?)**: Fields like `cgpa?` can be `undefined`, supporting beginners or graduates without crashing the UI.

2. Skill Entity & Prerequisite Graphs

```
export type SkillStatus = 'completed' | 'in-progress' | 'next' | 'locked'
export type SkillPriority = 'critical' | 'high' | 'recommended'

export interface Skill {
  id: string
  name: string
  category: SkillCategory
  description: string
  currentLevel: 'None' | 'Beginner' | 'Intermediate' | 'Advanced'
  requiredLevel: 'Beginner' | 'Intermediate' | 'Advanced'
  status: SkillStatus
  priority: SkillPriority
  prerequisites: string[] // Array of prerequisite skill IDs (DAG edges)
  depends: string[] // Array of dependent skill IDs
  estimatedWeeks: number
  whyItMatters: string
  resources?: ResourceLink[]
}
```

The `prerequisites: string[]` and `depends: string[]` arrays establish graph relationships without requiring complex pointer structures.

3. Generics: ApiResponse<T>

```
export interface ApiResponse<T> {
  data: T | null
  error: string | null
  loading: boolean
}
```

Generics Explained: `<T>` acts as a type placeholder. `ApiResponse<StudentProfile>` ensures `data` is typed as a student profile, while `ApiResponse<Skill[]>` guarantees an array of skills, avoiding duplicate response wrapper types.

08 State Management & Hooks Architecture

Waypoint uses native React state primitives and Context instead of heavy external state stores like Redux, keeping the architecture lean and transparent.



- `AuthContext` (Global): Holds `user` object and authentication methods (`login` , `signup` , `logout`). Restores session on mount from `localStorage` .
- `useNodesState` & `useEdgesState` (Canvas): Internal React Flow hooks. Handles dragging and viewport selections without re-rendering outer layouts.
- `useState` in Wizard (Local): Manages active step index (0 to 3) and partial profile object.
- `usePathname` (URL State): Listens to router changes to highlight active navigation tabs in the Sidebar.

09 Data Flow & Mock Data Inventory

Data Classification	Where It Resides	How It Changes	Current Reality in Codebase
UI State	<code>useState</code> , <code>usePathname</code>	User clicks, keystrokes, active tabs	Real Interactive State
Derived Data	<code>useMemo</code> , filtered arrays	Re-computed during render cycle	Real Calculations (progress %, filters)
Client Storage	Browser <code>localStorage</code>	Updated by Onboarding / Login	Simulated Persistence (keys: <code>waypoint_profile</code> , <code>waypoint_session</code>)
Simulated Data	<code>frontend/lib/mock-data/</code>	Read-only mock repositories	Simulated Repositories (skills, routes, destinations)
Backend Data	FastAPI / PostgreSQL	Network HTTP requests	Not Connected Yet (planned for next milestone)

10 Styling & Design System

Waypoint implements a purposeful, high-contrast design system inspired by tools like Linear and Notion.

1. Custom Tailwind Tokens (`tailwind.config.ts`)

Extends the standard theme with a dedicated navy palette used for sidebars and headers:

- `navy-900` (`#0F1629`) & `navy-950` (`#080d1a`): Used for sidebars and deep dark cards.
- `blue-600` (`#2563EB`): Primary brand accent, button backgrounds, and active ring borders.
- `emerald-500` (`#10B981`): Acquired skills, completed milestones, and origin start nodes.
- `amber-500` (`#F59E0B`): Career destination goal nodes and closing-soon deadline alerts.

2. The Safe Style Merging Helper: `cn()`

Located in `frontend/lib/utils.ts`, this utility prevents CSS class conflicts:

```
import { clsx, type ClassValue } from 'clsx'
import { twMerge } from 'tailwind-merge'

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

If a component defaults to `px-4 py-2` and a parent passes `className="px-6"`, `twMerge` replaces `px-4` with `px-6` instead of creating an invalid duplicate.

11 What Is Actually Complete? (Honest Audit)

A. Fully Implemented (Genuinely Functional)

- Multi-page routing across all 9 public and protected pages in Next.js 14 App Router.
- Application shell layout with fixed sidebar, active route indicators, and top breadcrumbs.
- Multi-step Onboarding wizard collecting and validating academic profile data.
- Client-side authentication flow (login, signup, logout) backed by browser storage.
- Route protection gatekeeper (`AuthGuard`) redirecting unauthenticated users to `/login`.
- Interactive React Flow DAG graph rendering with zoom, pan, minimap, and auto-centering.
- Custom node component (`SkillNode`) with distinct styles for Start, Goal, Completed, and Locked states.
- Dynamic topological layout algorithm (`generateDestinationGraph`) arranging nodes by prerequisite depth.
- Slide-over skill detail drawer (`SkillDetailPanel`) showing curated study resources.
- Opportunity Radar directory with keyword search, category tabs, and bookmark toggling.
- Complete set of design system primitives (`Button`, `Badge`, `Card`, `ProgressRing`, `ProgressBar`, `SkeletonLoader`, `EmptyState`, `ErrorState`).

B. Partially Implemented

- **Dashboard Analytics:** Computes progress percentage dynamically, but reads data from `localStorage` rather than an authenticated server session.
- **Route Comparison:** Renders 3 routes side-by-side, but route definitions come from static mock arrays.

C. Mocked / Simulated

- **Authentication:** Account records are stored in browser `localStorage`; passwords are base64-encoded via `btoa` rather than salted/hashed on a server.
- **Skill Registry & Prerequisites:** Reads from in-memory arrays in `frontend/lib/mock-data/skills.ts`.

D. Scaffolded (Future Placeholders)

- `frontend/components/interest-quiz/InterestQuiz.tsx`: Built for future elective discovery.
- `frontend/components/market-weather/MarketWeather.tsx`: Built for future live job market scraping.
- `frontend/components/crowd-trails/CrowdTrails.tsx`: Built for future alumni trajectory discovery.

12 Senior Code Quality & Engineering Review

1. [CRITICAL] Insecure Client Storage of Credentials

`frontend/lib/api.ts` stores accounts and passwords in browser `localStorage` using base64 obfuscation (`btoa`). In production, any Cross-Site Scripting (XSS) vulnerability or malicious browser extension can read `localStorage` . For Review 3, authentication must be migrated to HTTP-only, secure, `SameSite=Lax` JWT session cookies issued by FastAPI.

2. [HIGH] Direct Mock Data Imports Inside Feature Components

`components/route-map/RouteMap.tsx` directly imports `getDestinationById` from `@/lib/mock-data/destinations` and `getSkillById` from `@/lib/mock-data/skills` , bypassing the API layer. When connecting FastAPI, changing `lib/api.ts` alone will not update these components. All data requests must go through `lib/api.ts` .

3. [MEDIUM] Duplicate DAG Generation Implementations

`lib/mock-data/routes.ts` maintains static coordinate arrays for the AI/ML route alongside a dynamic generator (`generateDestinationGraph`) for other routes. Unifying all routes under the procedural DAG generator will eliminate redundant coordinate maintenance.

13 Recommended Learning Roadmap

To master, maintain, and extend this codebase independently as an IT student, follow this curriculum in order:

- JavaScript ES6+ & Async Programming:** Understand arrow functions, destructuring, Promises, and `async/await` .
Study: `frontend/lib/api.ts` , `frontend/lib/utls.ts`
- TypeScript Types & Interfaces:** Learn strict interfaces, union types (`|`), optional fields (`?`), and Generics (`<T>`).
Study: `frontend/lib/types.ts`
- React Core & Modern Hooks:** Master component state (`useState`), side effects (`useEffect`), memoization (`useCallback` , `React.memo`), and Context (`useContext`).
Study: `frontend/contexts/AuthContext.tsx` , `frontend/app/dashboard/page.tsx`
- Next.js 14 App Router:** Understand server vs client boundaries (`'use client'`), dynamic parameters (`[id]`), and layout hierarchies.
Study: `frontend/app/layout.tsx` , `frontend/app/route/[id]/page.tsx`
- Tailwind CSS & Design Systems:** Learn utility composition, custom color extensions, responsive prefixes, and conflict-free class merging via `cn()` .
Study: `frontend/styles/globals.css` , `frontend/components/ui/Button.tsx`
- React Flow Graph Engine:** Learn custom node authoring, handle attachment points, smoothstep edge routing, and viewport camera manipulation.
Study: `frontend/components/route-map/RouteMap.tsx` , `frontend/components/route-map/SkillNode.tsx`
- Graph Data Structures (DAGs):** Study topological sorting, recursion, cycle detection, and horizontal layer centering algorithms.
Study: `frontend/lib/mock-data/routes.ts` (function `generateDestinationGraph`)

14 Progressive Code Walkthrough

Let us study the codebase progressively, starting from the simplest helper utility and building up to the custom React Flow DAG engine.

Step 1: The Foundation Helper — `lib/utils.ts`

```
import { type ClassValue, clsx } from 'clsx'
import { twMerge } from 'tailwind-merge'

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

Explanation: This utility takes any number of class arguments, resolves conditional objects via `clsx`, and passes the result to `twMerge` to resolve Tailwind class precedence without string collisions.

Step 2: Reusable UI Primitive — `components/ui/Badge.tsx`

```
import { cn } from '@lib/utils'

type BadgeVariant = 'default' | 'success' | 'warning' | 'danger' | 'info' | 'neutral'

const variants: Record<BadgeVariant, string> = {
  default: 'bg-blue-50 text-blue-700 border-blue-100',
  success: 'bg-green-50 text-green-700 border-green-100',
  warning: 'bg-amber-50 text-amber-700 border-amber-100',
  danger: 'bg-red-50 text-red-700 border-red-100',
  info: 'bg-indigo-50 text-indigo-700 border-indigo-100',
  neutral: 'bg-slate-50 text-slate-600 border-slate-200',
}

export function Badge({ variant = 'default', children, className, dot }: BadgeProps) {
  return (
    <span className={cn('inline-flex items-center gap-1.5 px-2 py-0.5 text-xs font-medium rounded-full border', variants[variant], className)}>
      {dot && <span className="w-1.5 h-1.5 rounded-full bg-current" />}
      {children}
    </span>
  )
}
```

Explanation: A pure presentation component. `bg-current` ensures the inner dot matches the variant's text color automatically.

Step 3: Global Session Provider — `contexts/AuthContext.tsx`

```
export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<AuthUser | null>(null)
  const [isLoading, setIsLoading] = useState(true)

  useEffect(() => {
    const stored = getCurrentUser()
    setUser(stored)
    setIsLoading(false)
  }, [])

  const login = useCallback(async (email: string, password: string) => {
    const u = await authLogin(email, password)
    setUser(u)
  }, [])
  ...
}
```

Explanation: Restores user sessions from `localStorage` on initial mount. `useCallback` memoizes handler functions to prevent unnecessary child re-renders.

Step 4: Custom Graph Node — components/route-map/SkillNode.tsx

```
export const SkillNode = memo(function SkillNode({ data, selected }: NodeProps<RouteNodeData>) {
  if (data.isStart) {
    return (
      <div className="relative group">
        <Handle type="source" position={Position.Bottom} className="!bg-blue-600 !w-3 !h-3" />
        <div className="flex items-center gap-2 px-4 py-2.5 bg-[#0B0F19] text-white rounded-xl">
          <Navigation className="w-3.5 h-3.5 text-emerald-400" />
          <span className="text-xs font-bold">Current Profile</span>
        </div>
      </div>
    )
  }
  ...
  return (
    <div className="relative group">
      <Handle type="target" position={Position.Top} className={cn('!w-2.5 !h-2.5', isCompleted ? '!bg-emerald-500' : '!bg-blue-600')} />
      <div className={cn('w-[165px] rounded-2xl p-3.5 border-2', isCompleted ? 'border-emerald-300' : 'border-blue-500')}>
        <h4 className="text-xs font-bold text-slate-900">{data.label}</h4>
        <span className="text-[9px] text-slate-500">{data.category}</span>
      </div>
      <Handle type="source" position={Position.Bottom} className={cn('!w-2.5 !h-2.5', isCompleted ? '!bg-emerald-500' : '!bg-blue-600')} />
    </div>
  )
})
```

Explanation: Renders custom nodes inside React Flow. `<Handle />` anchors connection edges at `Position.Top` and `Position.Bottom`.

15

Complete End-to-End System Architecture Map

This master diagram illustrates the complete architectural pipeline from the user's browser down through the presentation components, API abstraction layer, and target backend services:



Conclusion & Review 2 Readiness: The Waypoint frontend is completely decoupled, responsive, and robustly architected. By establishing clean TypeScript interface contracts and standardizing all network operations behind `frontend/lib/api.ts`, the student team can seamlessly connect the FastAPI backend during Review 3 without redesigning any visual components or user workflows.